

Reporte del sistema de asignación de taxis

Diego Alberto Antonio Espinosa
A01736064@tec.mx

Dylan Issac Mancilla Lopez
A01739718@tec.mx

Carlos Fernando Velázquez Valles
A01384464@tec.mx

12 de junio de 2026

1. Introducción

En este proyecto implementamos un sistema sencillo de asignación de taxis usando Phoenix/Elixir en el backend y React en el frontend. La idea principal es simular el proceso de solicitar un viaje, contactar conductores, esperar respuestas y manejar cancelaciones.

Trabajamos con tres versiones separadas dentro del mismo repositorio:

- **v1**: versión base de clase.
- **v2**: versión donde se contactan tres conductores al mismo tiempo.
- **v3**: versión con cancelaciones, penalización, llegada del taxi y archivo de servicios no exitosos.

Decidimos manejar las versiones como carpetas independientes dentro del mismo repositorio. Esto nos ayudó a comparar el avance sin mezclar cambios entre versiones.

Repositorio: https://github.com/Diego-Alberto-A/taxi_26_elixir

2. Escenario v1

La primera versión corresponde a la implementación inicial hecha en clase. La dejamos como referencia base. Esta versión sirve para entender la estructura general del proyecto: un cliente manda una solicitud, el backend crea una reservación y se contacta a un conductor.

No modificamos esta versión porque representa el punto de partida del ejercicio.

3. Escenario v2

En v2 implementamos el escenario donde la aplicación contacta a tres conductores simultáneamente. El sistema selecciona tres candidatos, manda la misma solicitud a cada canal de conductor y espera una respuesta durante 1.5 minutos.

Si un conductor acepta, se notifica al cliente. Si nadie acepta dentro del tiempo, o si todos rechazan, se notifica que no fue posible asignar taxi.

3.1. Decisión de diseño

Usamos un `GenServer` por reservación. Esto nos pareció una buena opción porque cada viaje tiene su propio estado: conductores contactados, rechazos, timer y estado activo.

```
timer_ref = Process.send_after(self(), :timeout, 90_000)
```

Ese timer representa los 1.5 minutos del diagrama. Cabe resaltar que también fue incluido un botón de `GenServer`

4. Escenario v3

La tercera versión extiende v2 con cancelaciones y llegada del taxi. Esta versión cubre los casos principales del diagrama:

- El cliente cancela antes de que un conductor acepte. No hay penalización.
- El cliente cancela después de que un conductor aceptó, pero cuando faltan más de tres minutos para que llegue. No hay penalización.
- El cliente cancela cuando faltan tres minutos o menos para que llegue el taxi. Se aplica una penalización de 20 pesos.
- Si el taxi llega, el viaje se considera exitoso y se notifica el inicio del viaje.

4.1. Cancelación

El frontend muestra el botón de cancelación cuando ya existe una reservación aceptada por un conductor. El botón manda una acción al backend:

```
body: JSON.stringify({action: "cancel", username: props.username})
```

El backend recibe esa acción y delega la decisión al `GenServer` de la reservación. Ahí se revisa si la reservación sigue esperando aceptación o si el taxi ya va en camino.

4.2. Penalización

La penalización se decide con base en el tiempo restante para la llegada del taxi.

```
remaining_seconds = max(state.arrives_at - System.monotonic_time(:second), 0)  
remaining_minutes = remaining_seconds / 60
```

Si el tiempo restante es mayor a tres minutos, la cancelación no tiene costo. Si es de tres minutos o menos, se informa al cliente que existe una penalización de 20 pesos.

4.3. Contador de llegada

Cuando un conductor acepta, el backend inicia el timer de llegada de 5 minutos. También manda actualizaciones cada 30 segundos para que el frontend muestre el ETA: 5, 4.5, 4, 3.5, etc.

```
eta_timer_ref = Process.send_after(self(), :eta_tick, 30_000)
```

También agregamos un botón de prueba llamado *Time skip*. Este botón resta 45 segundos del contador real del backend. Lo usamos para probar más rápido los escenarios de llegada y penalización.

4.4. Cierre de tarjetas de conductor

Cuando un conductor acepta, las tarjetas de los otros conductores se eliminan. Para esto el backend manda un evento a todos los conductores contactados:

```
TaxiBeWeb.Endpoint.broadcast(  
  "driver:" <> driver_username,  
  "booking_request_closed",  
  %{bookingId: state.request["booking_id"], status: status}  
)
```

En el frontend, cada conductor escucha ese evento y oculta su tarjeta si corresponde a la misma reservación.

5. Archivo de servicios no exitosos

Para representar el paso de *Archive service* del diagrama, usamos un archivo de texto simple. No implementamos una base de datos porque el proyecto no la estaba usando y queríamos mantener la solución directa.

El archivo se genera en:

```
v3/taxi_be/service_log.txt
```

Ahí se guardan servicios que no terminaron exitosamente, por ejemplo:

- cancelación antes de aceptación;
- cancelación después de aceptación;
- cancelación tardía con penalización;
- timeout sin conductor;
- todos los conductores rechazaron.

6. Capturas de pantalla

Aquí se muestran algunas pruebas de ejecución del proyecto, específicamente de la versión 3.

6.1. Solicitud enviada por el cliente

Customer: galadriel	
Pickup address	
Tecnologico de Monterrey, campus Puebla, Mexico	
Drop off address	
Triangulo Las Animas, Puebla, Mexico	
SUBMIT	
We are processing your request	
Driver: frodo	
	Viaje de 'Tecnologico de Monterrey, campus Puebla, Mexico' a 'Triangulo Las Animas, Puebla, Mexico' ACCEPT REJECT
Driver: pippin	
	Viaje de 'Tecnologico de Monterrey, campus Puebla, Mexico' a 'Triangulo Las Animas, Puebla, Mexico' ACCEPT REJECT
Driver: samwise	
	Viaje de 'Tecnologico de Monterrey, campus Puebla, Mexico' a 'Triangulo Las Animas, Puebla, Mexico' ACCEPT REJECT

6.2. Aceptación de un conductor

Pickup address	Customer: galadriel
Tecnologico de Monterrey, campus Puebla, Mexico	
Drop off address	
Triangulo Las Animas, Puebla, Mexico	
SUBMITCANCEL BOOKINGTIME SKIP	
We are processing your request	
Tu taxi (frodo) está en camino. Tiempo estimado: 5 minutos.	
Driver: frodo	
Driver: pippin	
Driver: samwise	

6.3. Llegada del taxi

Pickup address	Customer: galadriel
Tecnologico de Monterrey, campus Puebla, Mexico	
Drop off address	
Triangulo Las Animas, Puebla, Mexico	
SUBMIT	
We are processing your request	
Tu taxi (frodo) ha llegado. ¡Buen viaje!	
Driver: frodo	
Driver: pippin	
Driver: samwise	

6.4. Cancelación con penalización

Pickup address	Customer: galadriel
Tecnologico de Monterrey, campus Puebla, Mexico	
Drop-off address	
Triangulo Las Animas, Puebla, Mexico	
SUBMIT	
We will process your cancellation	
Reservación cancelada con penalización de \$20. El taxi estaba a 1.2 minuto(s) de distancia.	
Driver: frodo	
Driver: pippin	
Driver: samwise	

7. Cómo ejecutar

Backend:

```
cd v3/taxi_be
mix deps.get
mix phx.server
```

Frontend:

```
cd v3/taxi_fe
npm install
npm run dev
```

El backend corre en el puerto 4000 y el frontend normalmente en el puerto 5173.

8. Reflexión

El problema parece simple al inicio, pero se vuelve interesante cuando hay varios eventos ocurriendo al mismo tiempo. Un conductor puede aceptar mientras otro rechaza, el cliente puede cancelar, y también existen timers que pueden cerrar el flujo.

Una decisión importante fue centralizar el estado de cada reservación en un **GenServer**. Esto nos permitió manejar timers, aceptación, cancelación y llegada del taxi en un solo lugar. También nos ayudó a evitar que varios conductores ganaran el mismo viaje.

También pensamos que registrar los servicios no exitosos es útil. Aunque aquí usamos un archivo de texto, por simplicidad, en una aplicación real esto debería guardarse en una base de datos para auditoría, soporte al cliente y análisis de fallos.

9. Conclusión

La versión final cumple con el flujo principal del diagrama. Contacta conductores, maneja aceptación, limpia tarjetas pendientes, muestra ETA, permite cancelación, aplica penalización cuando corresponde y registra los cierres no exitosos.

La solución todavía es sencilla, pero queda preparada para extenderse.